

Step 1: Install Dependencies

```
# Cell 1: Install required libraries
!pip install opencv-python dlib gradio numpy pandas
```

```
Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.13.0.92)
Requirement already satisfied: dlib in /usr/local/lib/python3.12/dist-packages (19.24.6)
Requirement already satisfied: gradio in /usr/local/lib/python3.12/dist-packages (6.19.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: anyio<5.0, >=3.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (4.14.0)
Requirement already satisfied: brotli<=1.1.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (1.2.0)
Requirement already satisfied: fastapi<1.0, >=0.115.2 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.115.2)
Requirement already satisfied: gradio-client==2.5.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (2.5.0)
Requirement already satisfied: groovy~=0.1 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.1.2)
Requirement already satisfied: hf-gradio<1.0, >=0.4.1 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.4.1)
Requirement already satisfied: httpx<1.0, >=0.24.1 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.28.1)
Requirement already satisfied: huggingface-hub<2.0, >=1.2.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (1.2.0)
Requirement already satisfied: Jinja2<4.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (3.1.6)
Requirement already satisfied: MarkupSafe<4.0, >=2.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (3.0.2)
Requirement already satisfied: orjson~=3.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (3.11.9)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from gradio) (26.2)
Requirement already satisfied: pillow<13.0, >=8.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (11.3.0)
Requirement already satisfied: pydantic<=3.0, >=2.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (2.13.1)
Requirement already satisfied: pydub<1.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.25.1)
Requirement already satisfied: python-multipart<1.0, >=0.0.18 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.0.18)
Requirement already satisfied: pytz>=2017.2 in /usr/local/lib/python3.12/dist-packages (from gradio) (2025.2)
Requirement already satisfied: pyyaml<7.0, >=5.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (6.0.3)
Requirement already satisfied: safehttpx<0.2.0, >=0.1.7 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.1.7)
Requirement already satisfied: semantic-version~=2.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (2.0.0)
Requirement already satisfied: starlette<2.0, >=1.0.1 in /usr/local/lib/python3.12/dist-packages (from gradio) (1.0.1)
Requirement already satisfied: tomlkit<0.15.0, >=0.12.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.12.0)
Requirement already satisfied: typer<1.0, >=0.12 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.25.1)
Requirement already satisfied: typing-extensions~=4.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (4.12.0)
Requirement already satisfied: uvicorn>=0.14.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.49.0)
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (from gradio-client==2.5.0->gradio) (2025.10.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.2)
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packages (from anyio<5.0, >=3.0->gradio) (3.10.1)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from fastapi<1.0, >=0.115.2->gradio) (0.4.2)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from fastapi<1.0, >=0.115.2->gradio) (0.0.2)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1.0, >=0.24.1->gradio) (2026.0.1)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1.0, >=0.24.1->gradio) (1.0.7)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1.0, >=0.24.1->gradio) (0.16.0)
Requirement already satisfied: click>=8.4.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0, >=1.2.0->gradio) (8.1.8)
Requirement already satisfied: filelock>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0, >=1.2.0->gradio) (3.10.0)
Requirement already satisfied: hf-xet<2.0.0, >=1.5.1 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0, >=1.2.0->gradio) (1.5.1)
Requirement already satisfied: tqdm==4.42.1 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0, >=1.2.0->gradio) (4.42.1)
Requirement already satisfied: annotated-types==0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<=3.0, >=2.0->gradio) (0.6.0)
Requirement already satisfied: pydantic-core==2.46.4 in /usr/local/lib/python3.12/dist-packages (from pydantic<=3.0, >=2.0->gradio) (2.46.4)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer<1.0, >=0.12->gradio) (1.5.4)
Requirement already satisfied: rich>=13.8.0 in /usr/local/lib/python3.12/dist-packages (from typer<1.0, >=0.12->gradio) (13.9.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=13.8.0) (3.0.0)
Requirement already satisfied: pygments<3.0.0, >=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=13.8.0) (2.18.0)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=13.8.0) (0.1.2)
```

Step 2: Download the Pre-trained Landmark Model

```
# Cell 2: Download and decompress the Dlib shape predictor model
!wget http://dlib.net/files/shape_predictor_68_face_landmarks.dat.bz2
!bzip2 -dk shape_predictor_68_face_landmarks.dat.bz2
print("Dlib model downloaded and extracted successfully!")

--2026-07-13 16:29:28-- http://dlib.net/files/shape_predictor_68_face_landmarks.dat.bz2
Resolving dlib.net (dlib.net)... 107.180.26.78
Connecting to dlib.net (dlib.net)|107.180.26.78|:80... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://dlib.net/files/shape_predictor_68_face_landmarks.dat.bz2 [following]
--2026-07-13 16:29:28-- https://dlib.net/files/shape_predictor_68_face_landmarks.dat.bz2
Connecting to dlib.net (dlib.net)|107.180.26.78|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 64040097 (61M)
Saving to: 'shape_predictor_68_face_landmarks.dat.bz2'

shape_predictor_68_ 100%[=====] 61.07M 45.4MB/s in 1.3s

2026-07-13 16:29:30 (45.4 MB/s) - 'shape_predictor_68_face_landmarks.dat.bz2' saved [64040097/64040097]

Dlib model downloaded and extracted successfully!
```

Step 3: Complete Unified Application Code

```

# Cell 3: Main Application Logic with 3-Column Single-Screen Layout
import os
import cv2
import dlib
import json
import numpy as np
import pandas as pd
import gradio as gr

# Initialize Dlib Computer Vision tools
face_detector = dlib.get_frontal_face_detector()
landmark_predictor = dlib.shape_predictor("shape_predictor_68_face_landmarks.dat")

# Unified Knowledge Base synthesized from Sources 1, 2, 4, 5, and 6
PHYSIOGNOMY_RULES = {
    "face": {
        "long": "Wood-Shaped Architecture: Indicates profound strategic planning capabilities, high persistence, and a commanding presence.",
        "oval": "Charming & Diplomatic: Expresses an emotionally balanced, sweet, and highly adaptive character. Ideal for roles requiring diplomacy.",
        "round": "Water-Shaped Architecture: Reflects radiant emotional warmth, deep sensitivity, a caring nature, and a fluid, adaptable mindset."
    },
    "nose": {
        "long": "Commanding Power: Signifies strong individual determination, high self-confidence, proud nature, and a focus on long-term goals.",
        "short": "Operational Flexibility: Reflects high spontaneity, quick decision-making metrics, and operational adaptability."
    },
    "eyes": {
        "large": "Luminous & Expansive: Denotes profound expressiveness, high spiritual empathy, independent creativity, and a broad perspective.",
        "small": "Concentrated Focus: Denotes sharp analytical prowess, intense powers of concentration, deep observation skills, and a meticulous nature."
    }
}

def extract_and_analyze_face(input_image):
    if input_image is None:
        return None, "🙄 Please upload a clear face image to begin the reading."

    image = cv2.cvtColor(input_image, cv2.COLOR_RGB2BGR)
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    faces = face_detector(gray)

    if len(faces) == 0:
        return None, "❌ Error: AI could not resolve facial boundaries. Please ensure your face is well-lit and centered."

    shape = landmark_predictor(gray, faces[0])
    landmarks = np.array([[shape.part(i).x, shape.part(i).y] for i in range(68)])

    # Compute baseline geometric distances
    face_width = float(landmarks[16][0] - landmarks[0][0])
    face_height = float(landmarks[8][1] - landmarks[27][1])
    nose_length = float(landmarks[33][1] - landmarks[27][1])
    left_eye_width = float(landmarks[39][0] - landmarks[36][0])
    right_eye_width = float(landmarks[45][0] - landmarks[42][0])
    eye_width_avg = (left_eye_width + right_eye_width) / 2.0

    if face_width <= 0 or face_height <= 0:
        return None, "❌ Error: Invalid facial geometry metrics calculated."

    face_ratio = face_height / face_width
    nose_ratio = nose_length / face_height
    eye_ratio = eye_width_avg / face_width

    if face_ratio >= 1.40:
        face_type = "long"
    elif face_ratio <= 1.18:
        face_type = "round"
    else:
        face_type = "oval"

    nose_type = "long" if nose_ratio >= 0.38 else "short"
    eye_type = "large" if eye_ratio >= 0.24 else "small"

    report = (
        f"🌟 AI REPORT 🌟\n"
        f"===== \n\n"
        f"🏗️ [Structure]: {face_type.upper()} FACE\n"
        f"📜 {PHYSIOGNOMY_RULES['face'][face_type]}\n\n"
        f"👃 [Nasal]: {nose_type.upper()} NOSE\n"
        f"📜 {PHYSIOGNOMY_RULES['nose'][nose_type]}\n\n"
        f"👁️ [Ocular]: {eye_type.upper()} EYES\n"
        f"📜 {PHYSIOGNOMY_RULES['eyes'][eye_type]}"
    )

    # Cyan/Neon tracking overlay
    annotated_image = input_image.copy()

```

```

    for (x, y) in landmarks:
        cv2.circle(annotated_image, (x, y), 3, (0, 255, 242), -1)

    return annotated_image, report

# Custom CSS (Kept exactly identical to your theme preferences, optimized margins)
custom_css = """
.gradio-container {
    background: linear-gradient(135deg, #090714 0%, #130f26 50%, #05030a 100%) !important;
    font-family: 'Segoe UI', Roboto, Helvetica, Arial, sans-serif;
    padding-bottom: 20px !important;
}
.custom-header {
    text-align: center;
    margin-bottom: 1rem;
    padding: 10px;
    border-bottom: 1px solid rgba(212, 175, 55, 0.2);
}
.custom-header h1 {
    color: #d4af37 !important;
    font-size: 2.2rem !important;
    font-weight: 800 !important;
    text-shadow: 0 0 15px rgba(212, 175, 55, 0.4);
    margin-bottom: 0.2rem;
}
.custom-header p {
    color: #a3a1cc !important;
    font-size: 1rem;
}
.gr-box, .gr-panel, .block {
    background-color: rgba(25, 20, 46, 0.6) !important;
    border: 1px solid rgba(212, 175, 55, 0.15) !important;
    border-radius: 12px !important;
    backdrop-filter: blur(8px);
    box-shadow: 0 8px 32px 0 rgba(0, 0, 0, 0.37) !important;
}
.gr-button-primary {
    background: linear-gradient(90deg, #d4af37 0%, #aa841b 100%) !important;
    color: #090714 !important;
    font-weight: bold !important;
    border: none !important;
    box-shadow: 0 0 15px rgba(212, 175, 55, 0.3) !important;
    transition: all 0.3s ease !important;
}
.gr-button-primary:hover {
    transform: translateY(-2px);
    box-shadow: 0 0 25px rgba(212, 175, 55, 0.6) !important;
}
footer {
    display: none !important;
}
"""

with gr.Blocks(
    theme=gr.themes.Default(primary_hue="amber", neutral_hue="slate"),
    css=custom_css
) as app:

    gr.HTML(
        """
        <div class="custom-header">
            <h1>👁️ LAKSHANA.AI</h1>
            <p>Ancient Vedic Samudrik Shastra Analytics Meets Modern Computer Vision</p>
        </div>
        """
    )

    # Restructured into a clean 3-Column horizontal row
    with gr.Row():
        with gr.Column(scale=1):
            input_img = gr.Image(
                label="1. Upload Profile Photo",
                type="numpy"
            )
            submit_btn = gr.Button(
                "➤ Run Analysis",
                variant="primary"
            )

        with gr.Column(scale=1):
            output_img = gr.Image(
                label="2. Neural Landmark Map"
            )

```

```
with gr.Column(scale=1):
    output_txt = gr.Textbox(
        label="3. Shastra Interpretation",
        lines=14,
        interactive=False
    )

submit_btn.click(
    fn=extract_and_analyze_face,
    inputs=[input_img],
    outputs=[output_img, output_txt]
)

app.launch(inline=True, share=True)
```

/tmp/ipykernel_2292/711266400.py:136: UserWarning: The parameters have been moved from the Blocks constructor to with gr.Blocks()

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()

* Running on public URL: <https://cd156b9c35a7691aa6.gradio.live>

This share link is temporary and will last for up to 1 week (best effort). For free permanent hosting and GPU upg



LAKSHANA.AI

Ancient Vedic Samudrik Shastra Analytics Meets Modern Computer Vision

1. Upload Profile Photo



Drop Image Here

- or -

Click to Upload



2. Neural Landmark Map



⚡ Run Analysis

Next steps: [Deploy to Cloud Run](#)